Университет ИТМО

Мегафакультет компьютерных технологий и управления

Факультет программной инженерии и компьютерной техники

Отчет
по лабораторной работе №2
«сервис-ориентированная архитектура»

Вариант 67208

Выполнили:

Кортыш Константин

Байрамгулов Мунир

Преподаватель

Цопа Евгений Алексеевич

Санкт-Петербург, 2026 г.

Задание

На основе разработанной в рамках лабораторной работы №1 спецификации реализовать два веб-сервиса и использующее их API клиентское приложение.

Требования к реализации и развёртыванию сервисов:

- Первый («вызываемый») веб-сервис должен быть реализован на фреймворке Spring MVC REST и развёрнут в окружении под управлением сервера приложений Payara.
- Второй веб-сервис должен быть реализован на фреймворке JAX-RS, развёрнут в окружении под управлением ещё одного экземпляра сервера приложений Payara и вызывать REST API первого сервиса.
- Для обоих сервисов необходимо реализовать все функции, задокументированные в API, в строгом соответствии со спецификацией.
- Доступ к обоим сервисам должен быть реализован по протоколу https с самоподписанным сертификатом сервера. Доступ к сервисам посредством http без шифрования должен быть запрещён.

Требования к клиентскому приложению:

- Клиентское приложение может быть написано на любом веб-фреймворке, который можно запустить на сервере helios.
- Приложение должно обеспечить полный набор возможностей, предоставляемых API обоих сервисов, включая сортировку, фильтрацию и постраничный вывод элементов коллекции.
- Приложение должно преобразовывать передаваемые сервисами данные в человеко-читаемый вид: параграф текста, таблицу и т.д.
- Клиентское приложение должно информировать пользователя об ошибках, возникающих на стороне сервисов, в частности о том, что сервису были отправлены невалидные данные.

Оба веб-сервиса и клиентское приложение должны быть развёрнуты на сервере helios.

Выполнение

Исходный код

https://github.com/msuny-c/soa

Каталог	Содержимое
`openapi/`	спецификации сервисов из ЛР1 и сборка Swagger UI
`services/worker-service/`	Worker Collection Service: Spring MVC 6, Spring Data JPA, Bean Validation
`services/hr-service/`	HR Service: JAX-RS, CDI, MicroProfile Rest Client
`services/site/`	клиент: React, Ant Design, TanStack Query
`.github/`	CI/CD и скрипты настройки доменов Payara на helios
`docker/`, `compose.yaml`	запуск всего стека в Docker

Диаграмма развёртывания

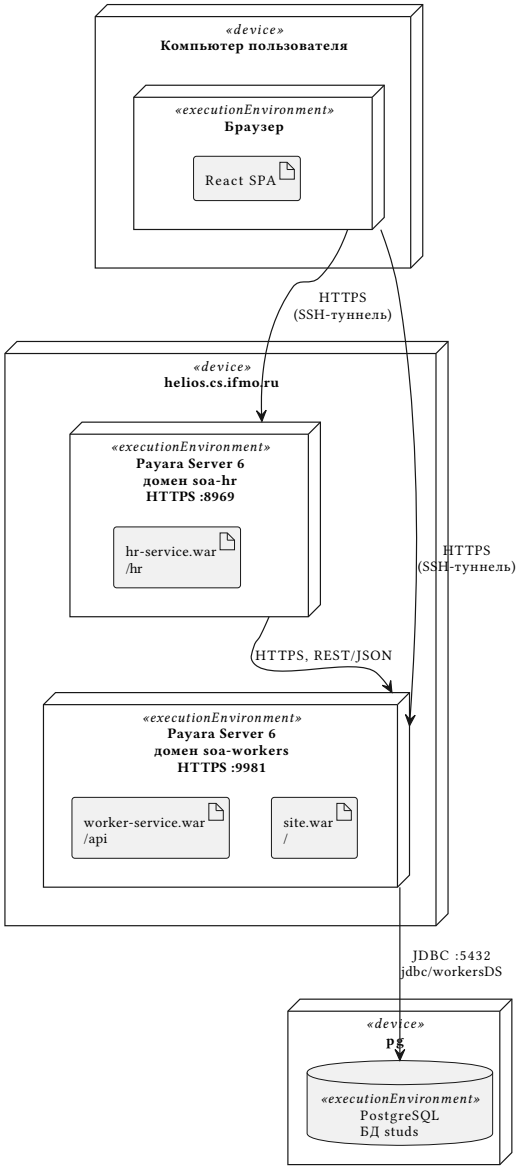


Рис. 1. Диаграмма развёртывания

Worker Collection Service

Spring MVC 6, Spring Data JPA, PostgreSQL. Контекст `/api`.

Метод	URL	Операция
`GET`	`/v1/workers`	список: сортировка, фильтрация, пагинация
`POST`	`/v1/workers`	добавление
`GET`	`/v1/workers/{id}`	получение по ID
`PUT`	`/v1/workers/{id}`	обновление
`DELETE`	`/v1/workers/{id}`	удаление
`GET`	`/v1/workers/salary/average`	средняя зарплата
`DELETE`	`/v1/workers/salary/value/{salary}`	удаление одного объекта с заданной зарплатой
`GET`	`/v1/workers/search/name/{substring}`	поиск по подстроке в имени

Параметры списка: `sort=-salary,name`, `filter=поле[операция]=значение` (операции `eq`, `ne`, `gt`, `gte`, `lt`, `lte`, `substr`, `null`), `page`, `size`. Коды ошибок: `400` — некорректный запрос, `404` — объект не найден, `409` — неуникальный `passportID`, `422` — нарушены ограничения полей.

Подключение к БД — JNDI-ресурс `jdbc/workersDS`, пул соединений настроен в Payara.

HR Service

JAX-RS, MicroProfile Rest Client. Контекст `/hr`.

Метод	URL	Операция
`POST`	`/index/{worker-id}/{coeff}`	индексация зарплаты сотрудника
`POST`	`/index/all/{org-id}/{coeff}`	индексация зарплат сотрудников организации

Адрес Worker Collection Service задаётся ключом `worker-service/mp-rest/url` в конфигурации домена. Коды ошибок: `400`, `404`, `422` — по спецификации, `502` — Worker Collection Service недоступен.

Клиентское приложение

React, Ant Design, TanStack Query. Типы и HTTP-клиент генерируются из OpenAPI-спецификаций (`openapi-typescript`, `openapi-fetch`). Разделы:

- Сотрудники — таблица с пагинацией, фильтрами, сортировкой по нескольким полям, созданием, редактированием и удалением;
- Отчёты и поиск — средняя зарплата, удаление по зарплате, поиск по имени;
- Индексация зарплат — операции HR Service.

Ошибки сервисов выводятся сообщениями, ошибки валидации — под полями формы.

HTTPS

- В обоих доменах отключён `http-listener-1`, работает только HTTPS (`http-listener-2`). Административный порт переведён на HTTPS.

- Для каждого домена `keytool` генерирует самоподписанный сертификат (SAN: `localhost`, `helios.cs.ifmo.ru`, `se.ifmo.ru`, `127.0.0.1`).
- Сертификат домена `soa-workers` импортирован в truststore домена `soa-hr`.

Выводы

По спецификации из ЛР1 реализованы Worker Collection Service на Spring MVC и HR Service на JAX-RS. Сервисы развёрнуты на helios в двух экземплярах Payara, доступны только по HTTPS с самоподписанными сертификатами. HR Service вызывает API первого сервиса через MicroProfile Rest Client. Клиент на React использует все операции обоих сервисов.